

# shroom: A Python framework for Ambisonics room acoustics simulation and binaural rendering

Yhonatan Gayer<sup>1</sup> and Boaz Rafaely<sup>1</sup>

<sup>1</sup> School of Electrical and Computer Engineering, Ben-Gurion University of the Negev, Israel

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

## Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [✉](#)

Submitted: 29 September 2026

Published: unpublished

## License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

## Summary

Spatial audio research for virtual and augmented reality, teleconferencing and hearing devices often represents sound fields in the Spherical Harmonics (SH) domain, known as Ambisonics. A typical study simulates a room, renders what a listener or a microphone array would capture in it, and processes those signals in the SH domain. We present SHroom (Spherical Harmonics ROOM), an open-source Python library that performs this whole workflow in one package, from room simulation to binaural rendering, head rotation, microphone-array simulation and Ambisonics encoding.

Existing tools cover either the room simulation or the downstream processing, so researchers bridge them with ad-hoc code that is hard to reproduce and compare. In SHroom, every step operates on one shared signal type through one processing interface, so a simulated room flows through the complete chain without format conversions. Built on the image-source engine of pyroomacoustics, SHroom reproduces the Ambisonic Room Impulse Response (ARIR) of its spherical-harmonic receivers while computing it about  $3\times$  faster for SH orders 4 to 12. SHroom is available at <https://github.com/Yhonatangayer/shroom> and installable via pip `install pyshroom`.

## Statement of need

The development and evaluation of spatial audio algorithms based on Ambisonics rely on the ability to simulate acoustic environments, process Ambisonics and other types of signals, and render binaural or loudspeaker signals. A typical research workflow encompasses diverse computational components: (i) the generation of ARIRs, (ii) the generation of Ambisonics signals by convolving dry audio signals with the ARIRs, (iii) SH-domain filtering, such as convolution with Head-Related Transfer Functions (HRTFs) or microphone array Acoustic Transfer Functions (ATFs) to produce signals at the ears or at the microphones, (iv) additional SH-domain processing including operations such as Wigner-D based listener head rotations, modeling of spherical microphone array prototypes, processing of SH-domain HRTFs, and encoding of Ambisonics from microphone arrays.

Implementing all these computational components within a single computation tool would streamline the research workflow. However, existing software can only provide implementations for various parts of such workflow.

## State of the field

For stages (i) and (ii), several existing room-acoustics simulators can generate ARIRs and convolve them with dry audio signals to produce Ambisonics signals. As summarized in Table

1, pyroomacoustics (Scheibler et al., 2018), MASP (Perez-Lopez & Politis, 2020), shoebox-roomsim (Politis, 2016), MCRoomSim (Wabnitz et al., 2010), and SAF (McCormack & Politis, 2024) generate ARIRs using the image-source method (ISM), whereas GSound-SIR (Zang & Kong, 2025) employs ray tracing (RT). These tools therefore cover room simulation and Ambisonics signal generation, but differ in the downstream processing they offer, as detailed in (iii) and (iv) above.

For stage (iii), support for SH-domain filtering, such as convolution with HRTFs or ATFs, is fragmented (see column ‘SH domain ATF processing’ in Table 1). Tools such as MASP (Perez-Lopez & Politis, 2020), SAF (McCormack & Politis, 2024), spaudiopy (Hold, 2025), sound-field-analysis-py, and pyfar/spharpy provide dedicated SH-domain ATF processing. In contrast, room simulators like shoebox-roomsim (Politis, 2016) and MCRoomSim (Wabnitz et al., 2010) do not provide post-simulation SH-domain ATF filtering.

Stage (iv) encompasses additional processing such as HRTF pre-processing (e.g. MagLS equalization), the application of SH rotation matrices for listener head tracking, modeling of open and rigid spherical microphone array prototypes, and Ambisonics array encoding. Specialized toolboxes such as spaudiopy (Hold, 2025) provide these capabilities, but omit room-acoustic engines altogether. Other packages cover only a subset of these features; for example, pyfar/spharpy lacks MagLS optimization, MASP (Perez-Lopez & Politis, 2020) and sound-field-analysis-py do not include native rotation matrices, and room simulators (pyroomacoustics, GSound-SIR, shoebox-roomsim, MCRoomSim) bypass stage (iv) entirely. While SAF (McCormack & Politis, 2024) supports a comprehensive feature set, it does so through standalone C/C++ modules rather than a unified, scriptable workflow. In conclusion, no toolbox is currently available which offers this wide range of computations, which may be important in spatial audio processing research.

## Contributions

To address the fragmented availability of tools within this spatial audio workflow, this paper introduces SHroom, an open-source Python library designed to unify room acoustics and SH processing. The main contributions of this work are:

- **An end-to-end SH-domain pipeline:** SHroom bridges the gap between room-acoustic simulation (stages i–ii) and downstream spatial processing (stages iii–iv). It is the first Python framework that allows users to generate ARIRs and seamlessly use them in advanced SH-domain processing without relying on external wrappers.
- **Comprehensive stage (iv) capabilities:** SHroom natively integrates SH processing. Each module implements an established method, all of which operate on the same ARIR and compose seamlessly with one another. Capabilities include Wigner-D rotation matrices, simulation of prototype and arbitrary microphone arrays, robust spatial encoding from arbitrary array geometries via the Ambisonics Signal Matching (ASM) (Gayer et al., 2024; McCormack et al., 2022) and Binaural Signal Matching (BSM) (Hermon et al., 2022; Madmoni et al., 2024), and HRTF preprocessing via MagLS (Kassakian, 2006; Lübeck et al., 2020; Schörkhuber et al., 2018) and array-aware MagLS HRTF (AA-MagLS) (Gayer et al., 2026).

**Table 1: Capability comparison of open-source spatial-audio software**

| Software        | Language | Room<br>sim. &<br>ARIR | SH<br>domain<br>ATF<br>processing |   | SH<br>rotation | SH<br>array<br>simulation | Array<br>encoding |
|-----------------|----------|------------------------|-----------------------------------|---|----------------|---------------------------|-------------------|
|                 |          |                        | MagLS                             |   |                |                           |                   |
| pyroomacoustics | Python   | ✓ (ISM)                | -                                 | - | -              | -                         | -                 |
| GSound-SIR      | Python   | ✓ (RT)                 | -                                 | - | -              | -                         | -                 |

| Software                    | Language      | Room<br>sim. &<br>ARIR | SH<br>domain<br>ATF<br>processing | MagLS    | SH<br>rotation | SH<br>array<br>simulation | Array<br>encoding |
|-----------------------------|---------------|------------------------|-----------------------------------|----------|----------------|---------------------------|-------------------|
| MASP                        | Python        | ✓ (ISM)                | ✓                                 | -        | -              | ✓                         | ✓                 |
| shoebox-roomsim             | MATLAB        | ✓ (ISM)                | -                                 | -        | -              | -                         | -                 |
| MCRoomSim                   | MATLAB        | ✓ (ISM)                | -                                 | -        | -              | -                         | -                 |
| SAF                         | C /<br>C++    | ✓ (ISM)                | ✓                                 | ✓        | ✓              | ✓                         | ✓                 |
| spaudiopy                   | Python        | -                      | ✓                                 | ✓        | ✓              | ✓                         | ✓                 |
| sound-field-<br>analysis-py | Python        | -                      | ✓                                 | -        | -              | ✓                         | ✓                 |
| pyfar / spharpy             | Python        | -                      | ✓                                 | -        | ✓              | ✓                         | ✓                 |
| <b>SHroom</b>               | <b>Python</b> | <b>✓ (ISM)</b>         | <b>✓</b>                          | <b>✓</b> | <b>✓</b>       | <b>✓</b>                  | <b>✓</b>          |

Note: ✓ = provided as a first-class feature; - = not provided. “Language” refers to the user-facing interface language of the software. “Room sim. & ARIR” means the simulated room field is exposed to the user as SH coefficients. “SH array simulation” means modal simulation of rigid or open spherical arrays with radial filters, not merely the placement of pressure microphones in a room. “Array encoding” means encoding those signals from arbitrary array geometries. SAF provides the individual capabilities as independent C modules rather than as one composable pipeline.

## Software design

SHroom is organised around one data type and one interface. `SpatialSignal` holds a multichannel signal together with its sampling rate, spatial grid and two domain flags (time or frequency; space or SH), and records every transform applied to it. Domain changes (`toFreq`, `toSH`, `toSpace`) and rotations (Wigner-D in the SH domain, or grid rotation in the space domain) are methods on this type, so every module accepts and returns the same object and checks the domain it needs rather than assuming it.

Processing steps (`BinauralDecoder`, `ArrayDecoder`, `ASMEncoder`, `BSMEncoder`) implement a single process method from `SpatialSignal` to `SpatialSignal`. They can therefore be applied in any order that is physically meaningful, and a `ProcessorChain` collapses a sequence of them into one equivalent filter by passing unit impulses through the chain once, so a long signal is convolved only a single time.

The room model deliberately reuses the `pyroomacoustics` image-source engine for geometry, wall materials and reflection bookkeeping instead of re-implementing it, and replaces only the receiver stage: all image sources are projected onto the SH basis in one vectorised evaluation and placed with the same fractional-delay filters, rather than simulating one directional microphone per SH channel. Frequency-dependent absorption is handled per octave band, as in `pyroomacoustics`. The trade-offs are explicit: SHroom supports shoebox rooms and the image-source method only, and it rejects `ray_tracing=True` instead of silently ignoring it; randomised image-source positions are enabled by default to reduce the metallic sound of the sparse late field, and `Room.ism_coverage()` warns when the reflection order is too low for the room’s reverberation time.

## Research impact statement

SHroom provides open and tested Python implementations of ASM (Gayer et al., 2024), BSM (Madmoni et al., 2024), MagLS (Schörkhuber et al., 2018), and AA-MagLS (Gayer et al., 2026),

112 together with functionality for Ambisonics and SH-domain processing. Two components have  
113 been validated against independent references. Using the same image-source engine, SHroom  
114 reproduces the ARIRs obtained with the spherical-harmonic receivers of pyroomacoustics to  
115 within the float32 precision of the latter, while computing in float64 and about  $3\times$  faster for  
116 SH orders  $4 \leq N \leq 12$ . Its BSM encoder has been validated against an independent MATLAB  
117 reference. The repository provides benchmark scripts for these comparisons and for evaluating  
118 the convergence of the ASM, spectrally equalized ASM, BSM, and AA-MagLS encoders with  
119 increasing SH order. It also includes six runnable examples covering the available binaural  
120 rendering paths: HRTF, MagLS, ASM, ASM with AA-MagLS, BSM, and head rotation.

121 SHroom is currently used by the authors in ongoing research on spatial audio and Ambisonics. It  
122 is also used by researchers within the authors' research group as part of their research workflows,  
123 including through its integration into the open-source AmbiDrop project (Tatarjitzky et al.,  
124 2026). Several ongoing research projects within the group rely on SHroom, with corresponding  
125 publications currently in preparation.

126 The authors have also received direct inquiries from researchers outside the research group  
127 regarding the use of SHroom in their research workflows. These interactions provide evidence of  
128 interest and use beyond the original development group. As further external research projects  
129 and publications become available, they will provide additional documented evidence of the  
130 adoption of SHroom.

131 SHroom is publicly distributed through PyPI (`pip install pyshroom`) and has been  
132 downloaded approximately 1,000 times since its first release in March 2026, according to  
133 pypistats, excluding mirrors. The project is maintained through continuous integration with a  
134 212-test suite covering Python 3.9–3.13, and provides contribution guidelines and citation  
135 metadata to support reuse and attribution.

## 136 AI usage disclosure

137 Generative AI tools were used to assist with software documentation, code refactoring, and the  
138 preparation of this paper (including editing for clarity and structure). All AI-suggested content  
139 was reviewed by the authors, and correctness was verified through the project's automated  
140 test suite and continuous-integration pipeline together with manual inspection; the algorithms,  
141 design decisions, and reported results are the authors' own. No generative AI was used to  
142 produce the experimental results.

## 143 Acknowledgements

144 The bundled HRTF data is the Neumann KU 100 spherical far-field HRIR compilation  
145 (Bernschütz, 2020). Room geometry and image-source computation build on pyroomacoustics  
146 (Scheibler et al., 2018), and the MagLS implementation follows the formulation of Schörkhuber  
147 et al. (Schörkhuber et al., 2018).

## 148 References

- 149 Bernschütz, B. (2020). *Spherical Far-Field HRIR Compilation of the Neumann KU100*. Zenodo.  
150 <https://doi.org/10.5281/zenodo.3928297>
- 151 Gayer, Y., Tourbabin, V., Ben-Hur, Z., Donley, J., & Rafaely, B. (2024). Ambisonics encoding  
152 for arbitrary microphone arrays incorporating residual channels for binaural reproduction.  
153 *2024 IEEE International Conference on Acoustics, Speech, and Signal Processing Workshops*  
154 *(ICASSPW)*, 244–248. <https://doi.org/10.1109/ICASSPW62465.2024.10627373>

- 155 Gayer, Y., Tourbabin, V., Ben-Hur, Z., Lou-Alon, D., & Rafaely, B. (2026). Array-aware  
156 ambisonics and HRTF encoding for binaural reproduction with wearable arrays. *IEEE*  
157 *Transactions on Audio, Speech and Language Processing*.
- 158 Hermon, S., Tourbabin, V., Ben-Hur, Z., Donley, J., & Rafaely, B. (2022). Binaural signal  
159 matching with an arbitrary array based on a sound field model. *Proceedings of the AES*  
160 *International Conference on Audio for Virtual and Augmented Reality (AVAR)*. <https://www.aes.org/e-lib/browse.cfm?elib=21861>  
161
- 162 Hold, C. (2025). *spaudiopy: Spatial audio python package*. [https://github.com/chris-](https://github.com/chris-hld/spaudiopy)  
163 [hld/spaudiopy](https://github.com/chris-hld/spaudiopy)
- 164 Kassakian, P. W. (2006). *Convex approximation and optimization with applications in*  
165 *magnitude filter design and radiation pattern synthesis* [PhD thesis]. University of California,  
166 Berkeley.
- 167 Lübeck, T., Helmholtz, H., Arend, J., Pörschmann, C., & Ahrens, J. (2020). Perceptual  
168 evaluation of mitigation approaches of impairments due to spatial undersampling in binaural  
169 rendering of spherical microphone array data. *Journal of the Audio Engineering Society*,  
170 68, 428–440. <https://doi.org/10.17743/jaes.2020.0038>
- 171 Madmoni, L., Ben-Hur, Z., Donley, J., Tourbabin, V., & Rafaely, B. (2024). Design and  
172 analysis of binaural signal matching with arbitrary microphone arrays. *arXiv Preprint*  
173 *arXiv:2408.03581*.
- 174 McCormack, L., & Politis, A. (2024). *Spatial\_Audio\_Framework: A cross-platform framework*  
175 *for developing spatial audio algorithms and software in C/C++*. [https://github.com/](https://github.com/leomccormack/Spatial_Audio_Framework)  
176 [leomccormack/Spatial\\_Audio\\_Framework](https://github.com/leomccormack/Spatial_Audio_Framework)
- 177 McCormack, L., Politis, A., Gonzalez, R., Lokki, T., & Pulkki, V. (2022). Parametric Ambisonic  
178 Encoding of Arbitrary Microphone Arrays. *IEEE/ACM Transactions on Audio, Speech, and*  
179 *Language Processing*, 30, 2062–2075. <https://doi.org/10.1109/TASLP.2022.3182857>
- 180 Perez-Lopez, A., & Politis, A. (2020). A Python library for multichannel acoustic signal  
181 processing. *148th Convention of the Audio Engineering Society*.
- 182 Politis, A. (2016). *Shoebbox room simulator for spatial sound processing*. [https://github.com/](https://github.com/polarch/shoebbox-roomsim)  
183 [polarch/shoebbox-roomsim](https://github.com/polarch/shoebbox-roomsim)
- 184 Scheibler, R., Bezzam, E., & Dokmanić, I. (2018). Pyroomacoustics: A python package for  
185 audio room simulation and array processing algorithms. *IEEE International Conference on*  
186 *Acoustics, Speech and Signal Processing (ICASSP)*, 351–355. [https://doi.org/10.1109/](https://doi.org/10.1109/ICASSP.2018.8461310)  
187 [ICASSP.2018.8461310](https://doi.org/10.1109/ICASSP.2018.8461310)
- 188 Schörkhuber, C., Zaunschirm, M., & Höldrich, R. (2018). Binaural rendering of ambisonic  
189 signals via magnitude least squares. *Proceedings of the DAGA*, 44, 339–342.
- 190 Tatarjitzky, M., Tourbabin, V., & Rafaely, B. (2026). AmbiDrop: Ambisonics-based array-  
191 agnostic neural speech enhancement. *arXiv Preprint arXiv:2607.00548*.
- 192 Wabnitz, A., Epain, N., Jin, C., & Schaik, A. van. (2010). Room acoustics simulation for  
193 multichannel microphone arrays. *Proceedings of the International Symposium on Room*  
194 *Acoustics (ISRA)*.
- 195 Zang, Y., & Kong, Q. (2025). *GSound-SIR: A spatial impulse response ray-tracing and*  
196 *high-order ambisonic auralization python toolkit*. <https://arxiv.org/abs/2503.17866>